Fachhochschule Nordwestschweiz
Hochschule für Technik

Convolutional Neural Nets (CNNs)

Deep Learning

FS 2026

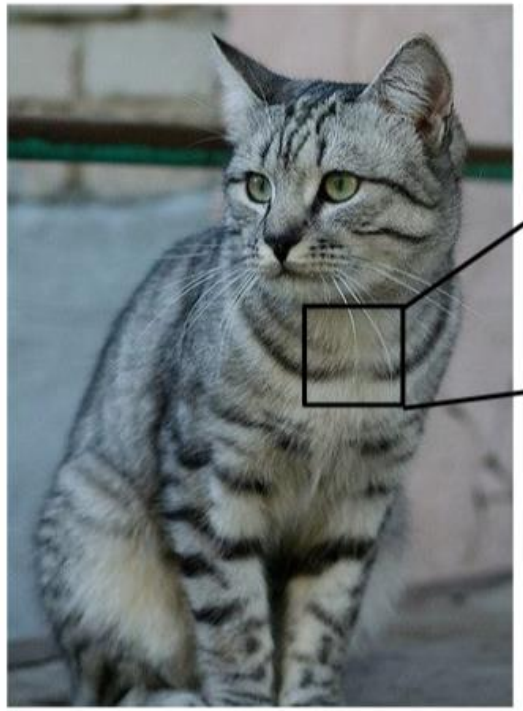
Martin Melchior

Agenda

- Need for Convolutions
- Conv Layer
- Classifiers
- GPU
- W&B

Need for Convolutions

Image representation



This image by Nikita is licensed under CC-BY 2.0

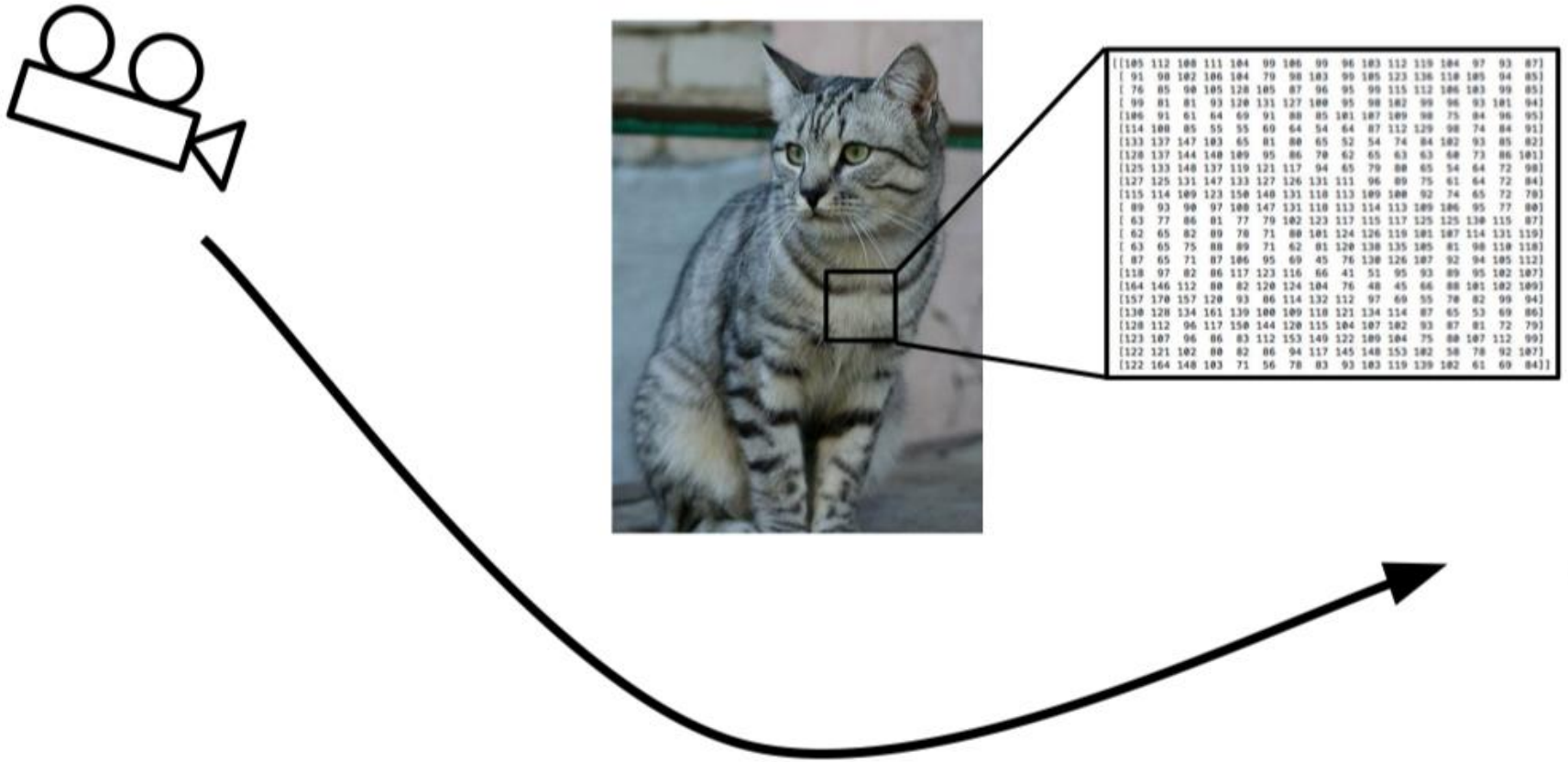
```
[[105 112 108 111 104  99 106  99  96 103 112 119 104  97  93  87]
 [ 91  98 102 106 104  79  98 103  99 105 123 136 110 105  94  85]
 [ 76  85  90 105 128 105  87  96  95  99 115 112 106 103  99  85]
 [ 99  81  81  93 120 131 127 100  95  98 102  99  96  93 101  94]
 [106  91  61  64  69  91  88  85 101 107 109  98  75  84  96  95]
 [114 108  85  55  55  69  64  54  64  87 112 129  98  74  84  91]
 [133 137 147 103  65  81  80  65  52  54  74  84 102  93  85  82]
 [128 137 144 140 109  95  86  70  62  65  63  63  60  73  86 101]
 [125 133 148 137 119 121 117  94  65  79  80  65  54  64  72  98]
 [127 125 131 147 133 127 126 131 111  96  89  75  61  64  72  84]
 [115 114 109 123 150 148 131 118 113 109 100  92  74  65  72  78]
 [ 89  93  90  97 108 147 131 118 113 114 113 109 106  95  77  80]
 [ 63  77  86  81  77  79 102 123 117 115 117 125 125 130 115  87]
 [ 62  65  82  89  78  71  80 101 124 126 119 101 107 114 131 119]
 [ 63  65  75  88  89  71  62  81 120 138 135 105  81  98 110 118]
 [ 87  65  71  87 106  95  69  45  76 130 126 107  92  94 105 112]
 [118  97  82  86 117 123 116  66  41  51  95  93  89  95 102 107]
 [164 146 112  80  82 120 124 104  76  48  45  66  88 101 102 109]
 [157 170 157 120  93  86 114 132 112  97  69  55  70  82  99  94]
 [130 128 134 161 139 100 109 118 121 134 114  87  65  53  69  86]
 [128 112  96 117 150 144 120 115 104 107 102  93  87  81  72  79]
 [123 107  96  86  83 112 153 149 122 109 104  75  80 107 112  99]
 [122 121 102  80  82  86  94 117 145 148 153 102  58  78  92 107]
 [122 164 148 103  71  56  78  83  93 103 119 139 102  61  69  84]]
```

What the computer sees

An image is just a big grid of numbers between [0, 255]:

e.g. 800 x 600 x 3
(3 channels RGB)

Challenges: viewpoint

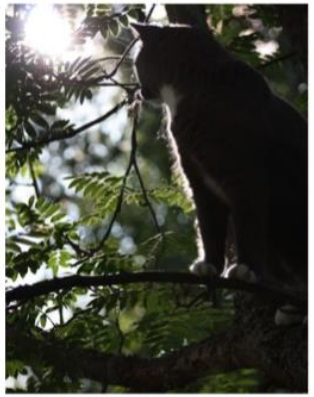


Challenges: illumination / deformations

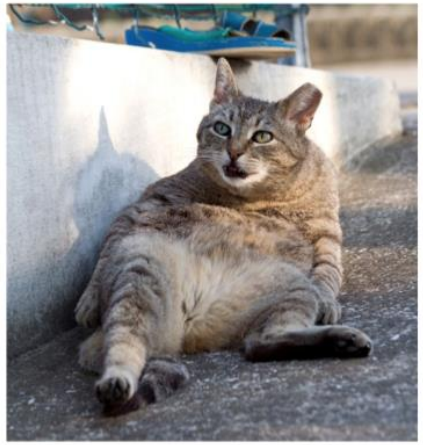
This image is CC0 1.0 public domain

This image is CC0 1.0 public domain

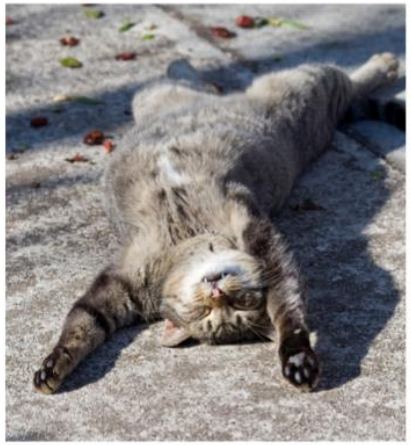
This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

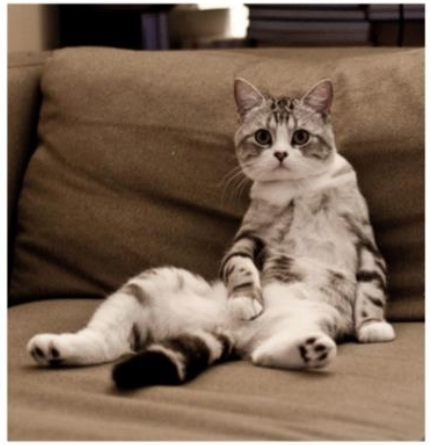


This image by Umberto Salvagnin
is licensed under CC-BY 2.0



This image by Umberto Salvagnin
is licensed under CC-BY 2.0

This image by sare bear is
licensed under CC-BY 2.0



This image by Tom Thai is
licensed under CC-BY 2.0

Challenges: occlusions / background

This image is CC0 1.0 public domain

This image is CC0 1.0 public domain

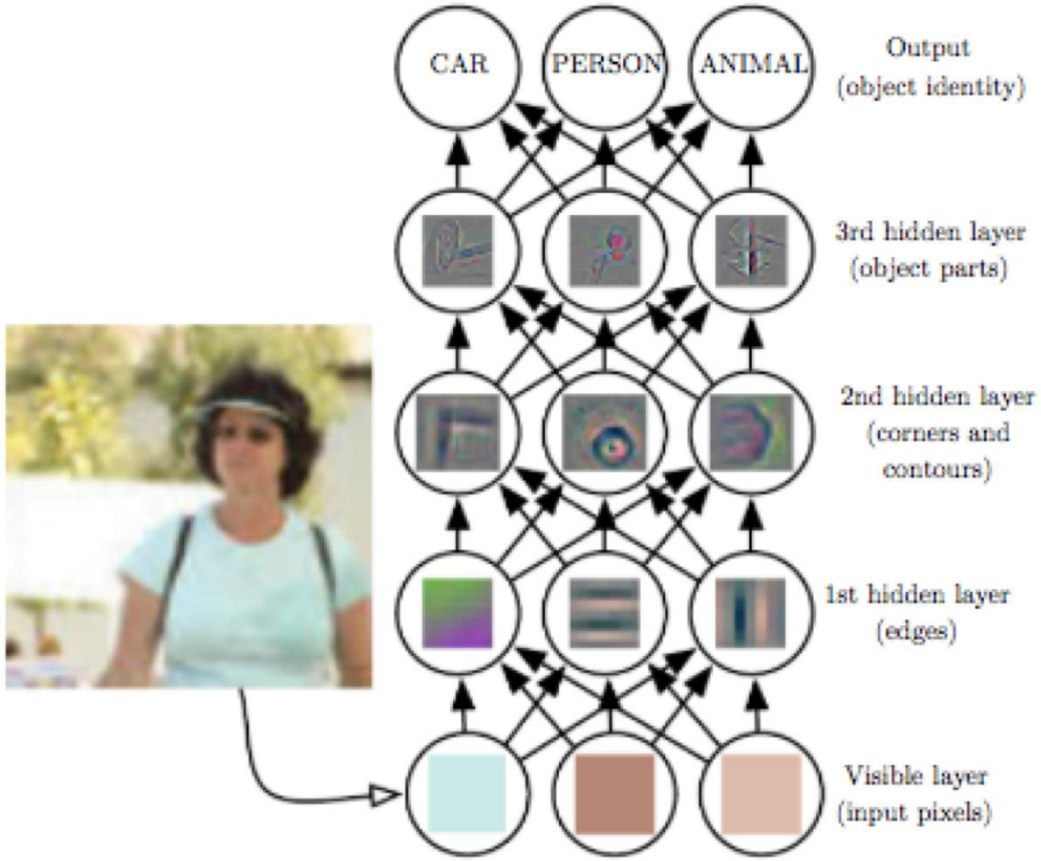
This image by jonsson is licensed under CC-BY 2.0

This image is CC0 1.0 public domain

This image is CC0 1.0 public domain

CNNs – Feature Extraction from Image Data

- Neighboring pixels share locality information in an image – such as color, contours (edges), etc.
- Convolutional Filters are designed to extract features from a set of neighboring pixels in images.
- Several filters can operate in parallel (extracting different features).
- Resulting conv layers can be stacked.

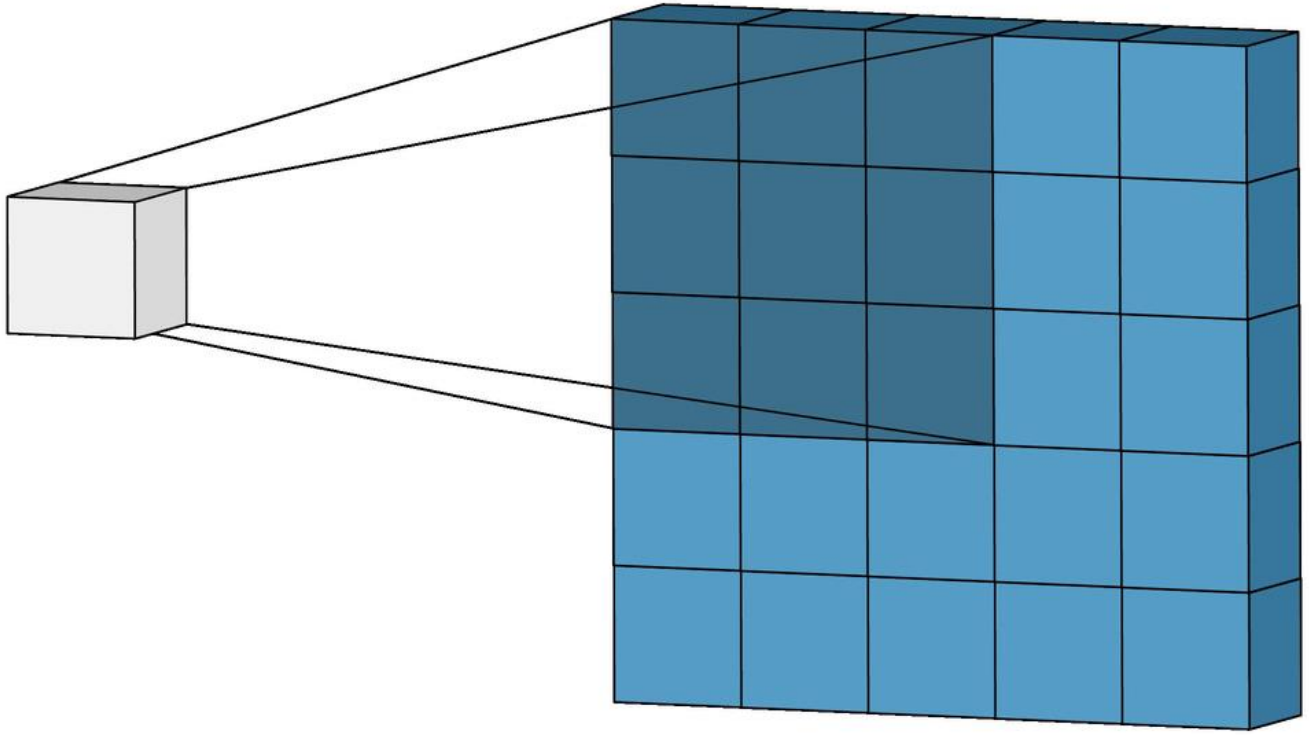


Convolutions Capture Local Spatial Structure

In a *dense* layer, the information on the spatial structure is completely lost as the output neurons are fully connected. The network could be trained with all pixels shuffled randomly (equally for all images) and the resulting network would perform the same.

Convolutional layers capture patterns reflecting the spatial structure in the images. They are learned and are applicable independent of the location of the pattern(s) in the image, in this sense are translation invariant.

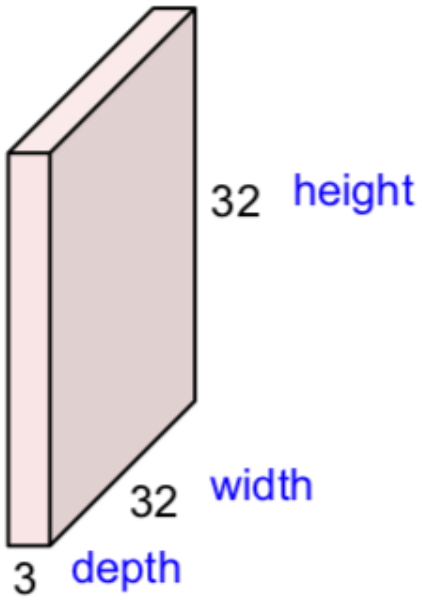
Conv Layer



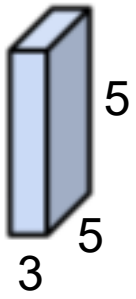
Convolution Layer - 2D Signals

Example with an RGB image

image 32x32x**3**
"3D matrix of pixels"



filter 5x5x**3**
"3D matrix of weights"



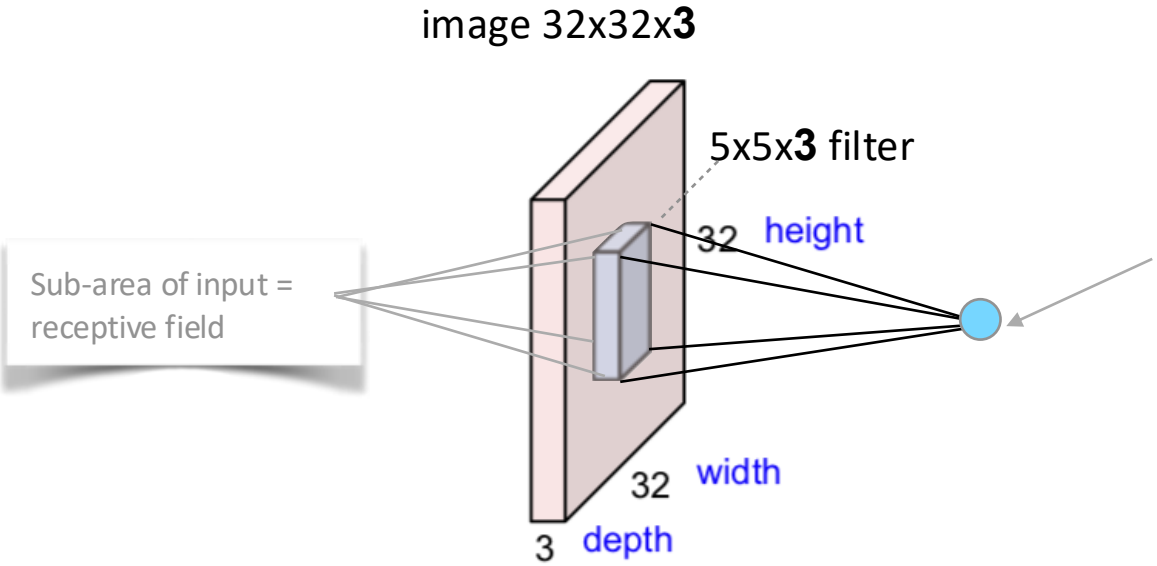
Filters have the same depth as the input "volume"

Filters are also called "kernels"

Images in colour have 3 "channels", Red-Green-Blue RGB

Convolution Layer - 2D Signals

The filter is convolved with the image, i.e. the filter is slided over the image spatially and, in each position, the dot products of the weights of the filter with the pixels values in the image at the filter position ("receptive field") is computed.



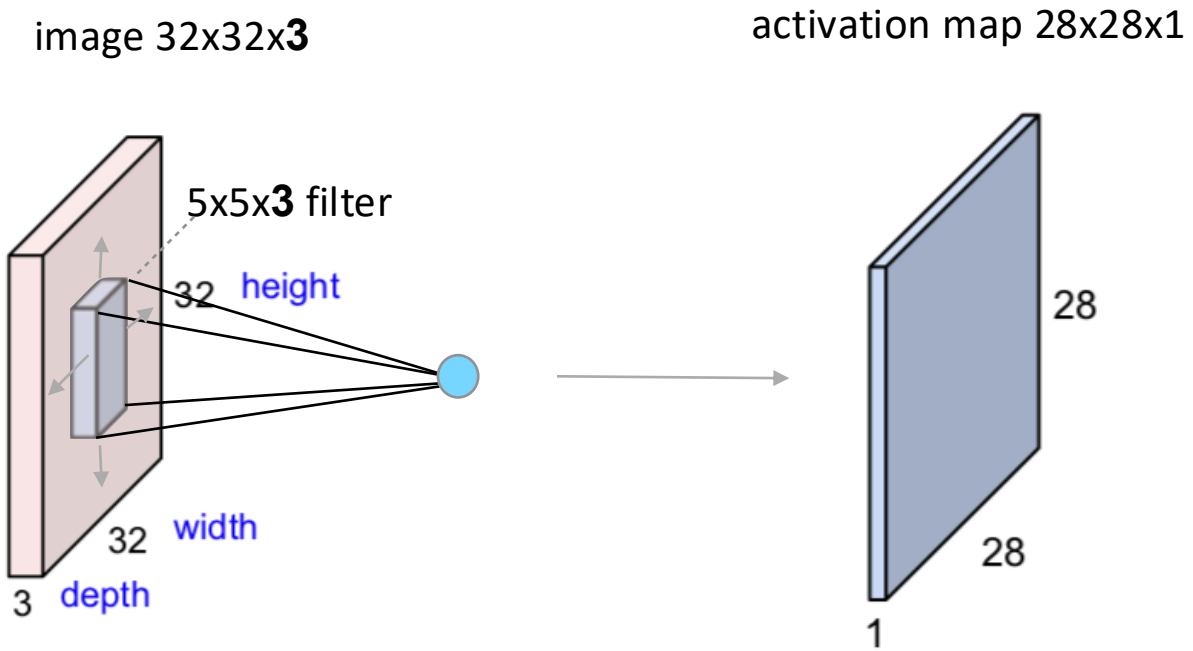
one number
result of computing the dot product between the weights of the filters and a small 5x5x3 chunk of the image;
sum of 5*5*3 products of floats, one addition of bias

Convolution Layer - 2D Signals

Result of convolving the filter with the image is called *activation map*.

This sliding of the filter gives us *translation equivariance* regarding to what the filter is sensitive for: Translating features in the input leads to same activation values at the translated position.

The operation *preserves the spatial structure* of the input.

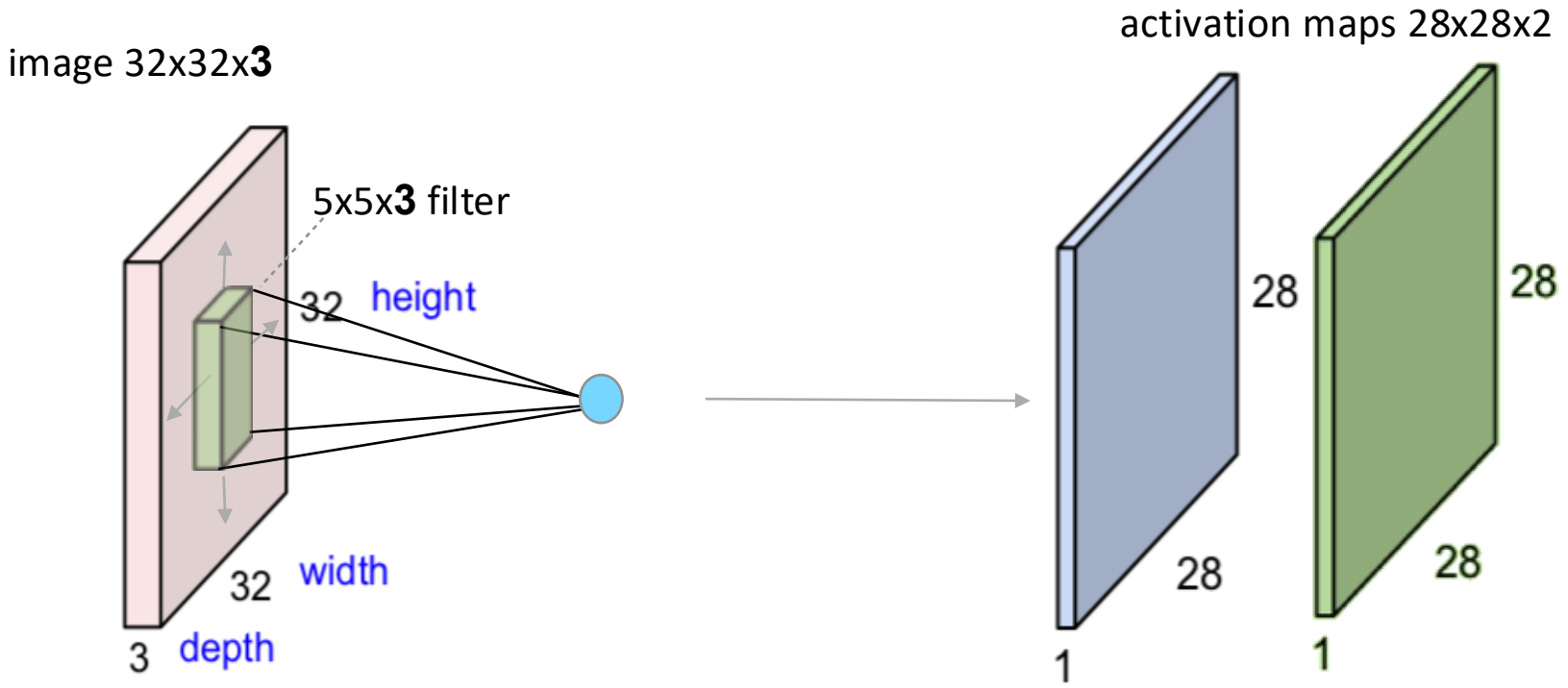


Why 28x28?

Convolution Layer - 2D Signals

Several filters producing several activation maps.

Applying a second filter (green) leads to a second activation map:

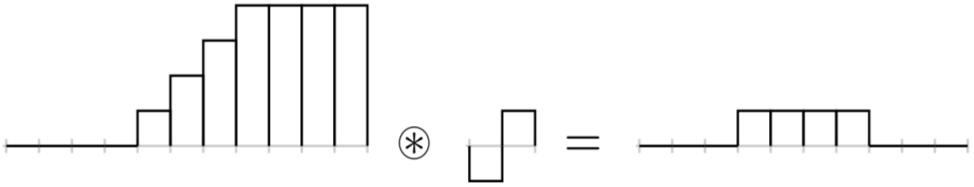


Convolutional layer consisting of *n* filters leads to an output with *n* activation maps.

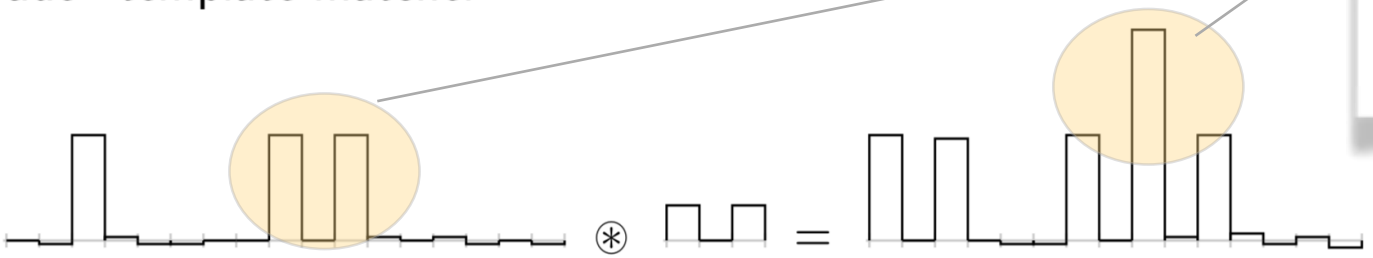
Convolution Layer - 1D Signals

Convolution can implement a differential operator

$$(0,0,0,0,1,2,3,4,4,4,4) \circledast (-1,1) = (0,0,0,1,1,1,1,0,0,0).$$

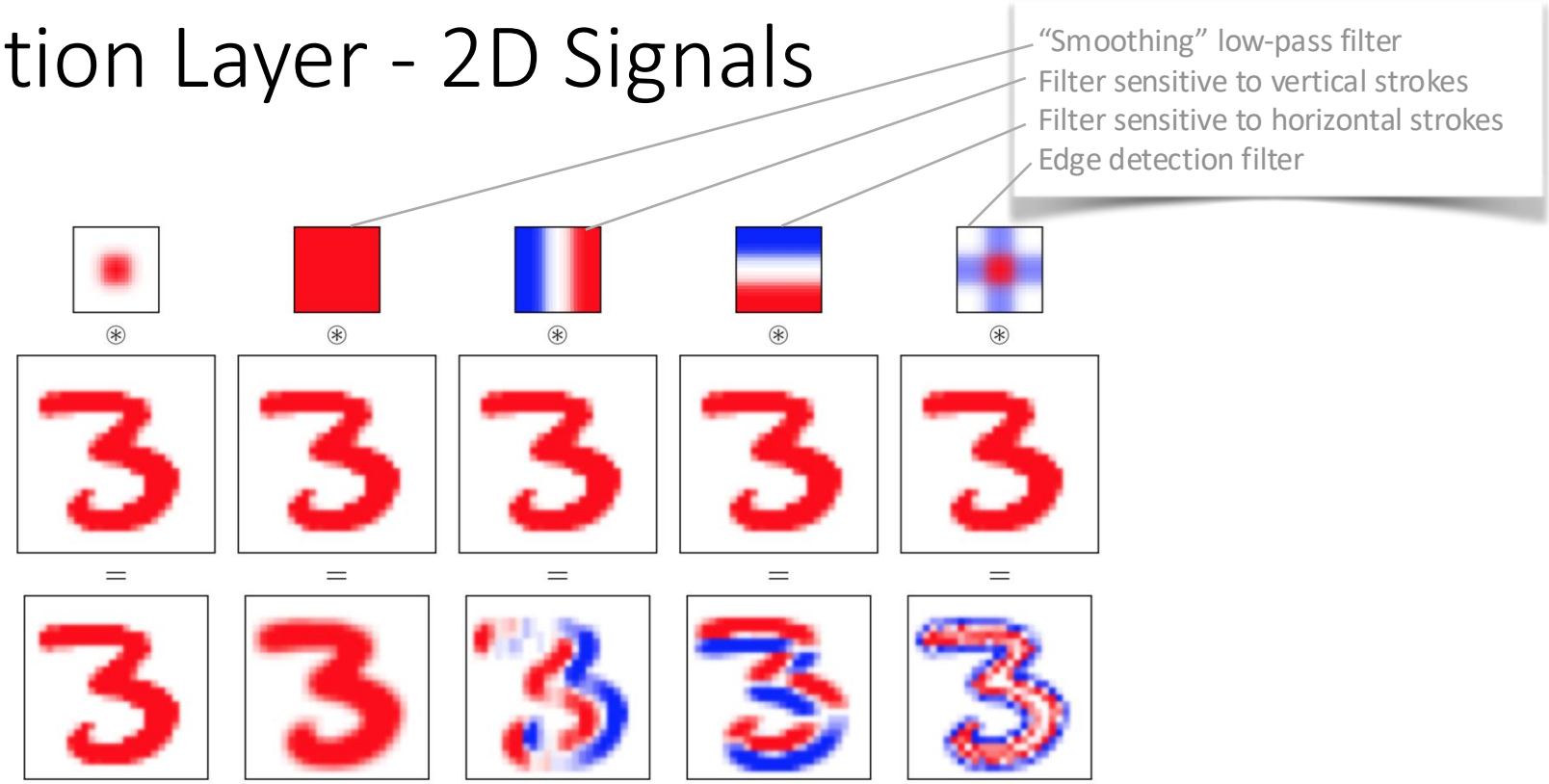


or a crude "template matcher"



The sub-area of the input that "matches" the shape of the filter produces the highest output.

Convolution Layer - 2D Signals

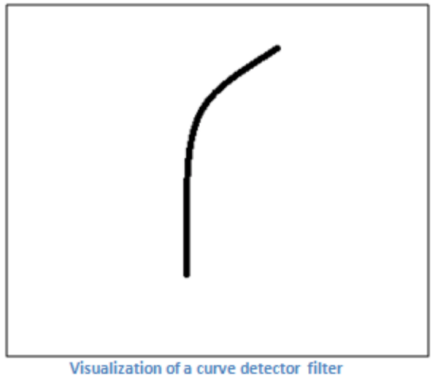


For a given problem, the "best" filters will be discovered through the training process, i.e. weights of the filters will be learnt

Convolution Layer - 2D Signals

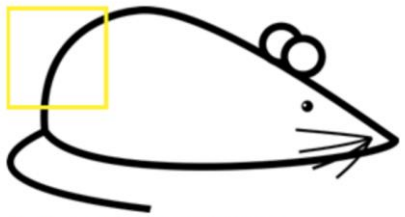
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

Original image



Visualization of the filter on the image

Visualization of the receptive field

0	0	0	0	0	0	30
0	0	0	0	50	50	50
0	0	0	20	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0
0	0	0	50	50	0	0

Pixel representation of the receptive field

*

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

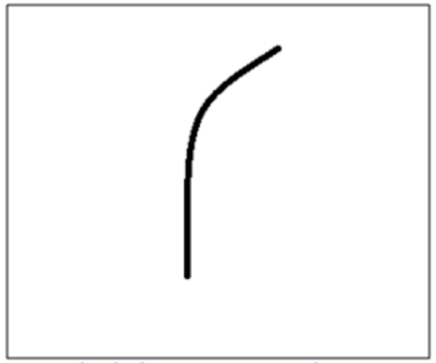
Multiplication and Summation = (50*30)+(50*30)+(50*30)+(20*30)+(50*30) = 6600 (A large number!)

Image: Adit Deshpande, https://adeshpande3.github.io/adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks-Part-2/

Convolution Layer - 2D Signals

0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter



Visualization of a curve detector filter

Visualization of the filter on the image

0	0	0	0	0	0	0
0	40	0	0	0	0	0
40	0	40	0	0	0	0
40	20	0	0	0	0	0
0	50	0	0	0	0	0
0	0	50	0	0	0	0
25	25	0	50	0	0	0

Pixel representation of receptive field

*

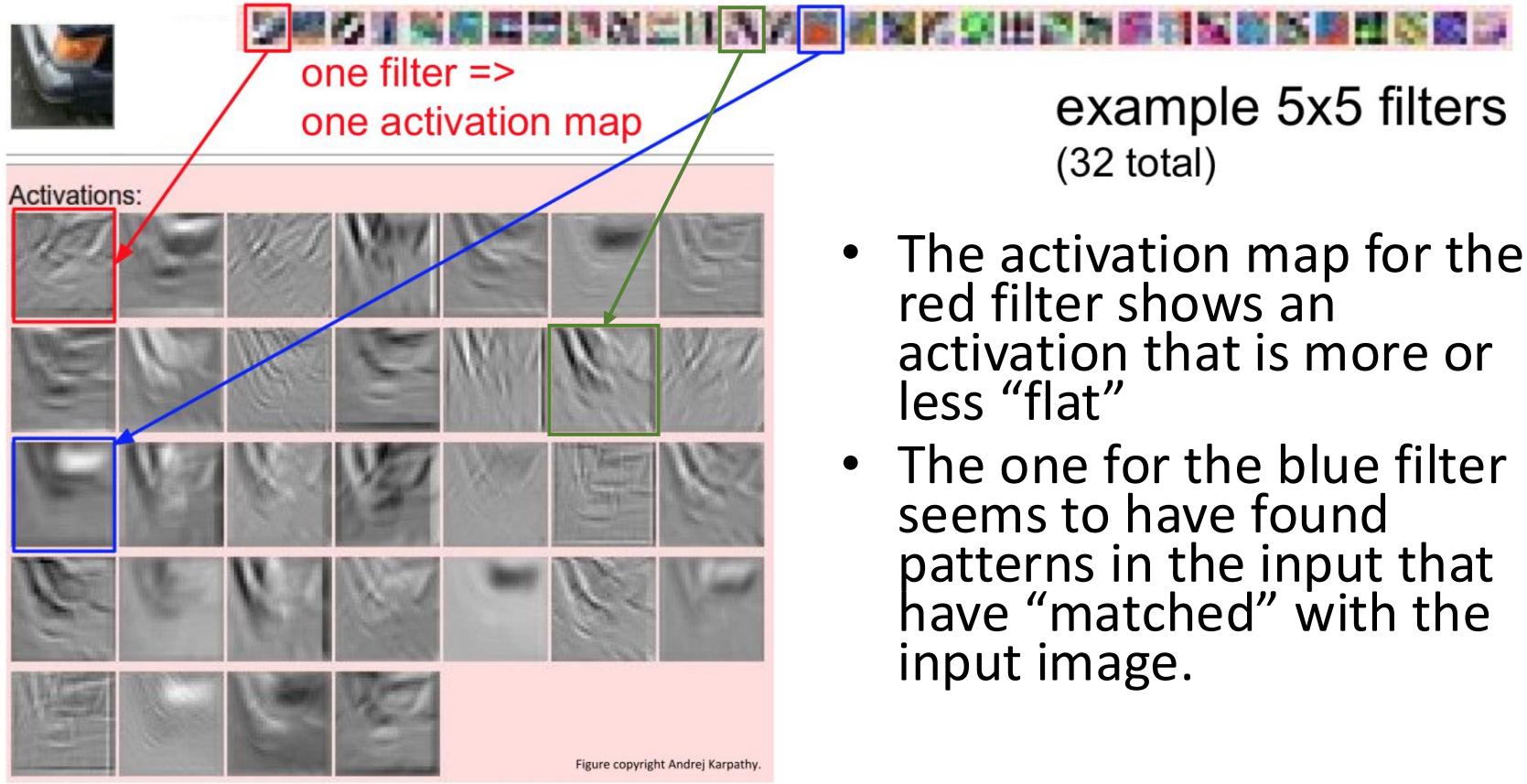
0	0	0	0	0	30	0
0	0	0	0	30	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	30	0	0	0
0	0	0	0	0	0	0

Pixel representation of filter

Multiplication and Summation = 0

Image: Adit Deshpande, https://adeshpande3.github.io/adeshpande3.github.io/A-Beginner's-Guide-To-Understanding-Convolutional-Neural-Networks-Part-2/

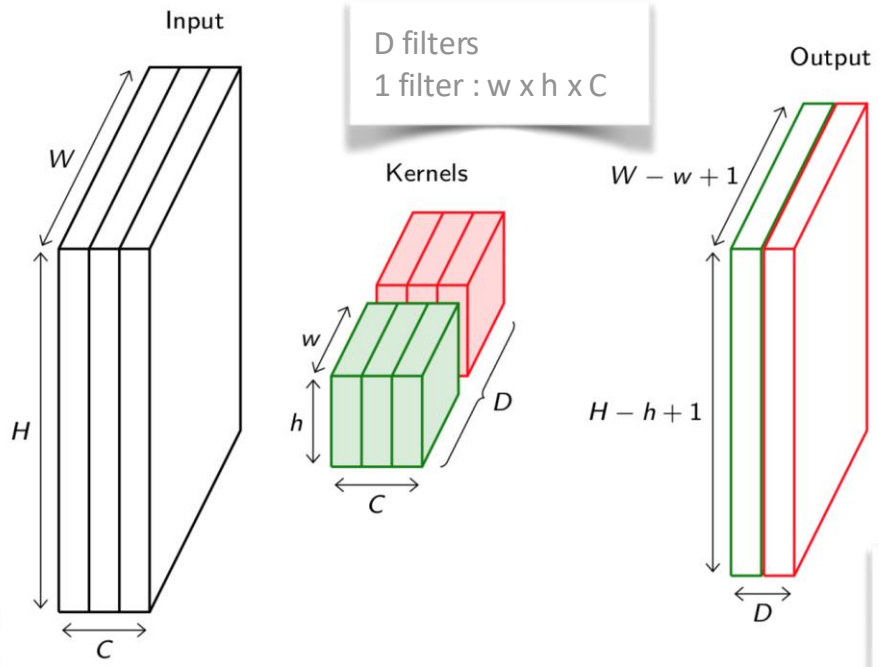
Convolution Layer - 2D Signals - CIFAR10



- The activation map for the red filter shows an activation that is more or less “flat”
- The one for the blue filter seems to have found patterns in the input that have “matched” with the input image.

Convolution layer - 2D signals - channels

- The input images have usually 3 channels, e.g. RGB
- The filters have then the same depth as the input “volume”
- The activation maps have a depth equal to the number of filters



D filters
1 filter : w x h x C

C channels
1 image : W x H x C

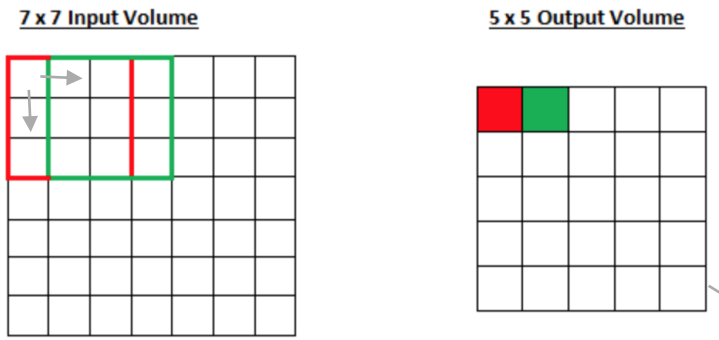
D activation maps = **depth**
1 map : (W-w+1) x (H-h+1) x D

Convolution layer - stride

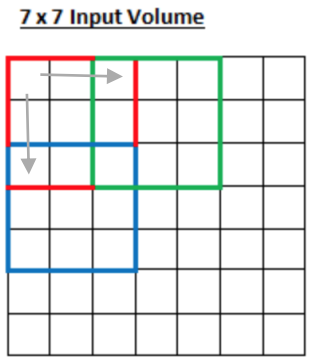
Stride S specifies a step size when moving the filter across the signal.

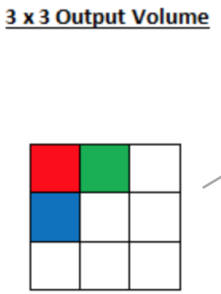
Rationale: Reduce computation and output size – no need of strong overlap.

S = 1



S = 2



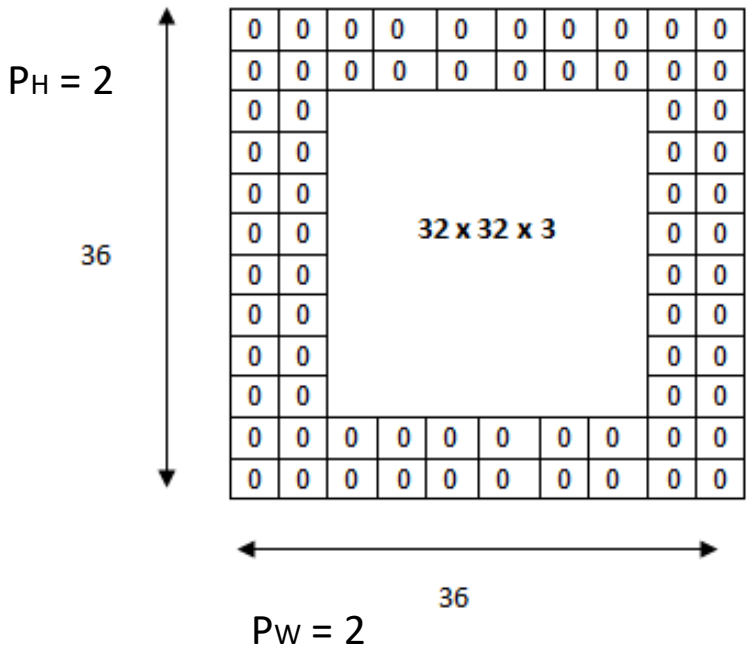


Larger stride S means less overlap and reduction of the activation map size (= smaller output volume).

Convolution layer - padding

Padding P specifies the size of a zeroed frame added around the input.

Rationale: Apply filter at the border of the input; avoid reducing the output dim.



The input volume is 32 x 32 x 3. If we imagine two borders of zeros around the volume, this gives us a 36 x 36 x 3 volume. Then, when we apply our conv layer with our three 5 x 5 x 3 filters and a stride of 1, then we will also get a 32 x 32 x3 output volume.

Condition to obtain an output size equal to the input size:

$$P_W = \frac{w-1}{2} \qquad P_H = \frac{h-1}{2}$$

w = width of filter
h = height of filter

Common settings: K=32, 64, 128, 512
h=w=3, S=1, P=1 h=w=5, S=1, P=2
h=w=1, S=1, P=0

Convolution layer - output size / # params

Output size:

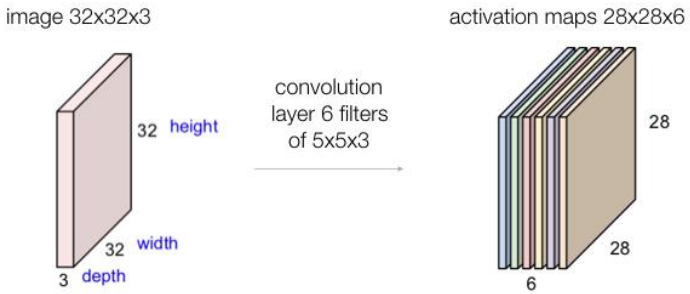
$$O_W = \frac{W - w + 2P_W}{S_w} + 1 \qquad O_H = \frac{H - h + 2P_H}{S_h} + 1$$

(W, H) = width and height of input
(w, h) = width and height of filter
P = padding, S = stride

Number of parameters:

$$N_{param} = \underbrace{whCD}_{\text{weights}} + \underbrace{D}_{\text{bias}}$$

C = number of channels
D = number of filters



5*5*3*6 + 6 =
456 parameters

Red

Green

Blue

Input Volume (+pad 1) (7x7x3)

x[:,:,0]

0	0	0	0	0	0	0
0	2	1	0	2	2	0
0	2	0	1	2	1	0
0	0	2	1	2	0	0
0	0	0	1	0	2	0
0	1	2	0	1	0	0
0	0	0	0	0	0	0

x[:,:,1]

0	0	0	0	0	0	0
0	0	2	2	1	0	0
0	0	2	2	1	1	0
0	2	0	2	0	1	0
0	1	1	0	2	0	0
0	0	0	1	2	0	0
0	0	0	0	0	0	0

x[:,:,2]

0	0	0	0	0	0	0
0	0	2	0	0	1	0
0	2	1	1	1	1	0
0	2	0	0	2	1	0
0	0	2	0	2	2	0
0	2	2	1	2	1	0
0	0	0	0	0	0	0

Filter W0 (3x3x3)

w0[:,:,0]

1	1	0
0	0	-1
1	1	-1

w0[:,:,1]

0	-1	1
0	-1	1
0	0	1

w0[:,:,2]

0	0	-1
0	0	-1
0	0	-1

Bias b0 (1x1x1)

b0[:,:,0]

1

Filter W1 (3x3x3)

w1[:,:,0]

1	0	1
1	1	1
0	1	-1

w1[:,:,1]

0	-1	1
0	1	0
-1	1	0

w1[:,:,2]

0	1	1
-1	1	1
1	1	1

Bias b1 (1x1x1)

b1[:,:,0]

0

Output Volume (3x3x2)

o[:,:,0]

3	-3	4
-1	-5	4
-5	0	3

o[:,:,1]

10	5	8
14	16	8
9	9	2

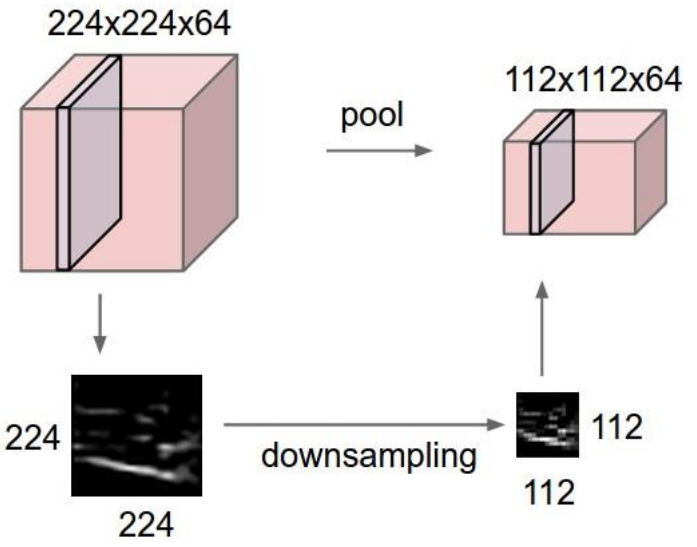
toggle movement

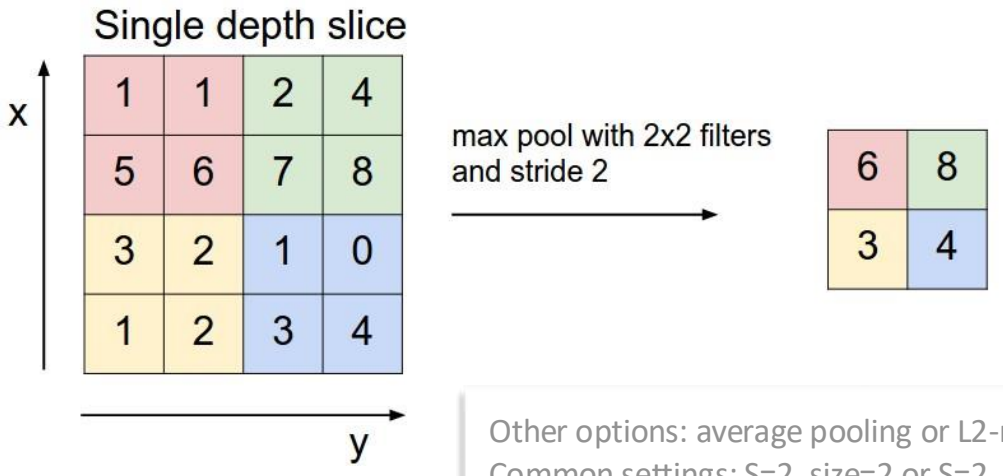
Max pooling layer

Reduces the spatial size of the representation.

Applies independently to every depth, defined by a stride and size.

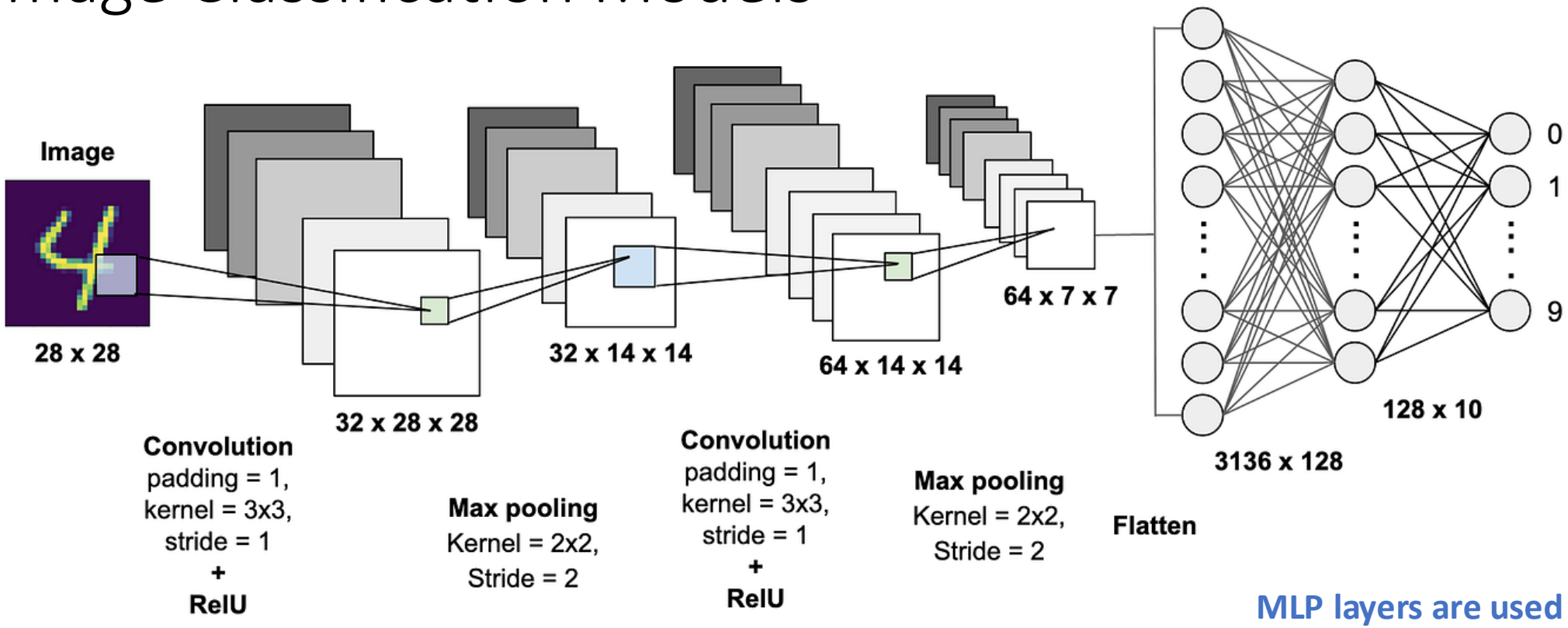
Most significant activations are kept, brings hierarchical approach; reduce the amount of computation and control overfitting.





Other options: average pooling or L2-norm pooling.
Common settings: S=2, size=2 or S=2, size=3

Image Classification Models



CNN and max/avg pooling layers for image feature extraction. Filters correspond to features and are learned.

MLP layers are used to find combinations of image features (extracted by CNN layers) that are best suited for the classification.

Hands-On

GPU

Bringing Execution to GPU in PyTorch

```
x = torch.randn((2,2))
x, x.device
```

```
(tensor([[-0.5562,  1.0103],
         [ 0.6685,  1.2575]]), device(type='cpu'))
```

any torch tensor lives on a device, see ‘device’ attribute

```
[116] torch.cuda.is_available()
```

```
True
```

check on your system whether you have access to gpu

```
[117] device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
      device
```

```
device(type='cuda')
```

get the device qualifier

```
[118] x = x.to(device)
      x, x.device
```

```
(tensor([[-0.5562,  1.0103],
         [ 0.6685,  1.2575]], device='cuda:0'), device(type='cuda', index=0))
```

move tensor to device for processing: creates a copy if it does not yet exist or returns the object if it exists.

Bringing Execution to GPU in PyTorch

```
[119] torch.cuda.device_count()
```

```
1
```

to check if you have more than one GPU

```
[121] for i in range(torch.cuda.device_count()):
        x = x.to("cuda:%i"%i)
        print(x.device)
```

```
cuda:0
```

use 'cuda:i' with i=0,1,… if you have more than one GPU

```
[120] torch.cuda.get_device_name(0)
```

```
'Tesla T4'
```

Important to know: You can't combine tensors living on different devices in operations; first move them on the same device.

```
RuntimeError: Expected all tensors to be on the same device, but found at least two devices, cpu and cuda:0!
```

Bringing Execution to GPU in PyTorch

Define model as usual,
i.e. agnostic w.r.t. available devices.

```
nepochs = 5


model = cnn_model()
cost_ce = torch.nn.CrossEntropyLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(nepochs):
    model.train()
    size = len(train_dataloader.dataset)
    nbatches = len(train_dataloader)
    cost = 0.0
    for batch, (X, Y) in enumerate(train_dataloader):

        pred = model(X)
        loss = cost_ce(pred, Y)
        cost += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    cost /= nbatches
    print("Epoch %i: %f"%(epoch, cost))
```

Move model and cost fct to device - define the computational graph for which we compute forward() and backward()

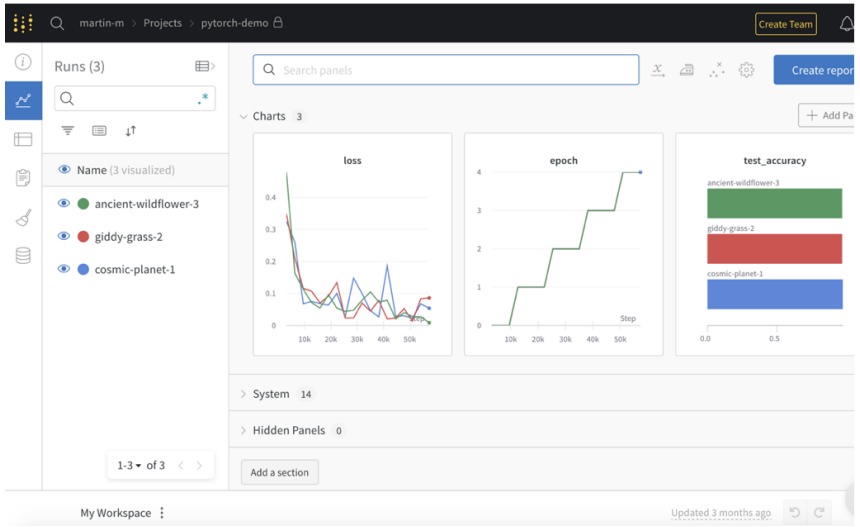
```
nepochs = 5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

model = cnn_model().to(device)
cost_ce = torch.nn.CrossEntropyLoss().to(device)
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)

for epoch in range(nepochs):
    model.train()
    size = len(train_dataloader.dataset)
    nbatches = len(train_dataloader)
    cost = 0.0
    for batch, (X, Y) in enumerate(train_dataloader):
        X, Y = X.to(device), Y.to(device)
        pred = model(X)
        loss = cost_ce(pred, Y)
        cost += loss.item()

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

    cost /= nbatches
    print("Epoch %i: %f"%(epoch, cost))
```

Move the data to the device.

Hands-On

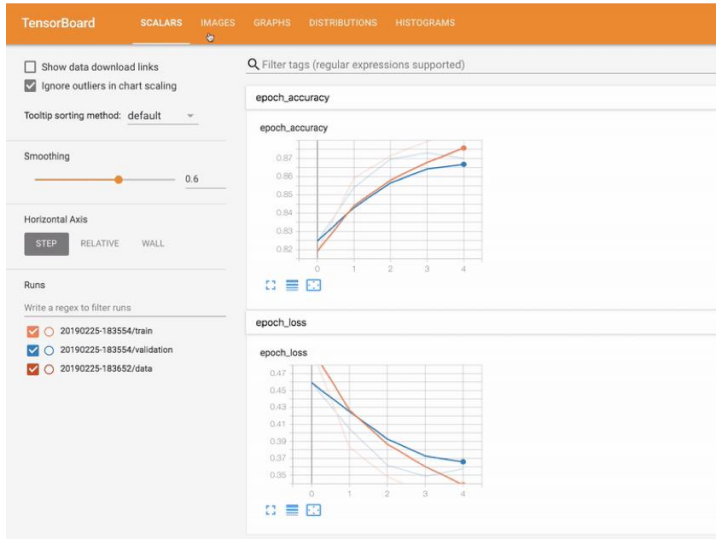
Monitoring Tools

Weight & Biases (WandB)



[pytorch-with-wandb]

Tensorboard



[pytorch-with-tensorboard]

Fachhochschule Nordwestschweiz
Hochschule für Technik

W&B

Communication with cloud service.
Requires login and needs to be initialised.

Configuration (hyper-parameters) need to be specified - facilitates navigation on web pages in the analysis of the results.

pip install wandb

```
# WandB – Import the wandb library
import wandb

# WandB – Login to your wandb account so you can log all your metrics
!wandb login
```

```
wandb: Currently logged in as: martin-m. Use `wandb login --relogin` to
```

```
# WandB – Initialize a new run
wandb.init(entity="martin-m", project="wandb-demo-ss23")
wandb.watch_called = False # Re-run the model without restarting the runtime,
```

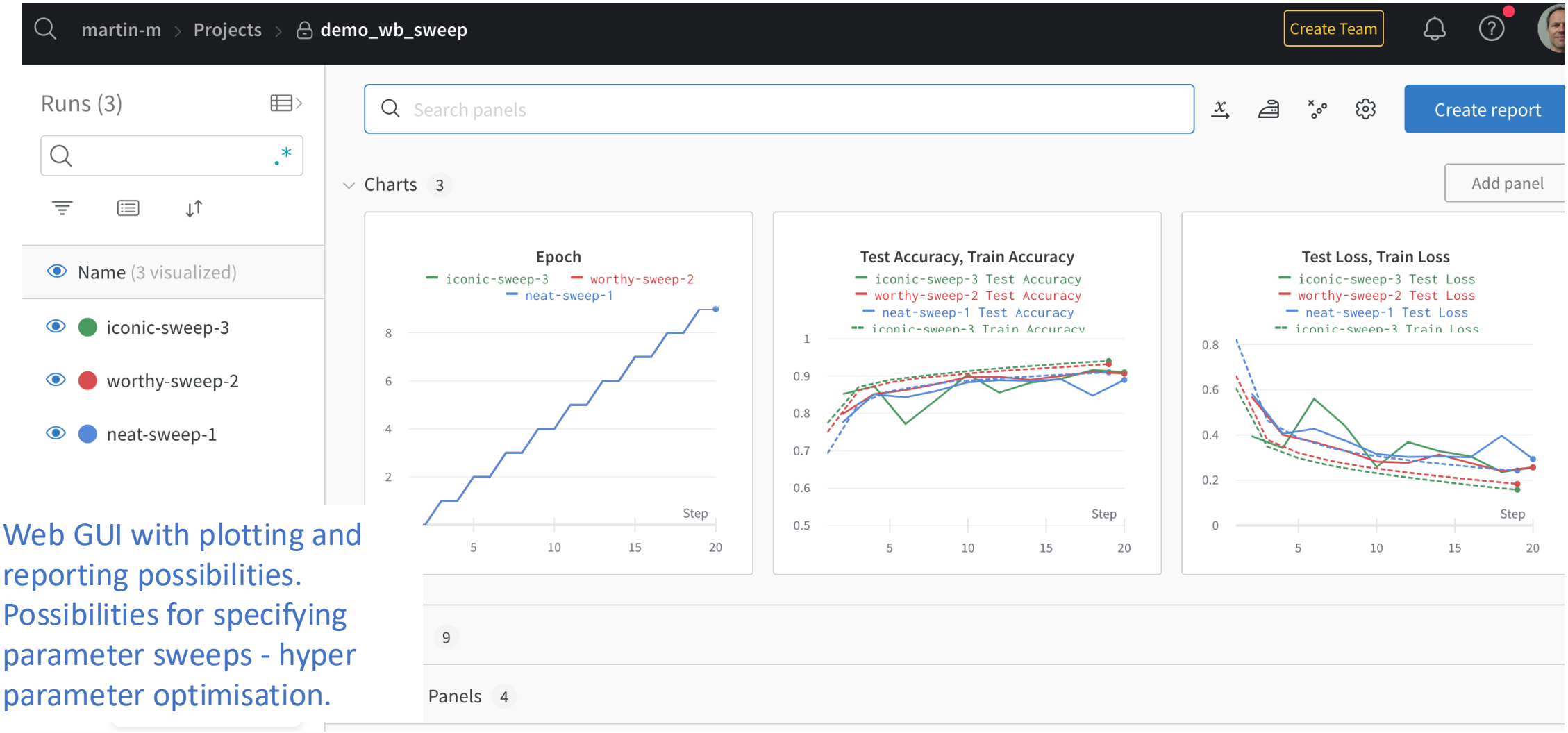
```
# WandB – Config is a variable that holds and saves hyperparameters and input
config = wandb.config          # Initialize config
config.batch_size = 64
config.nepochs = 10
config.lr = 0.1
config.log_interval = 10
```

```
def test(model, cost_ce, loader, epoch):
    model.eval()
    with torch.no_grad():
        size = len(loader.dataset)
        nbatches = len(loader)
        cost, acc = 0.0, 0.0
        for X, Y in loader:
            pred = model(X)
            cost += cost_ce(pred, Y).item()
            acc += (pred.argmax(dim=1) == Y).type(torch.float).sum().item()
        cost /= nbatches
        acc /= size
        print("Validate epoch %i : %6.3f , %6.3f"%(epoch, cost, acc))
        wandb.log({"Epoch": epoch, "Test Accuracy": acc, "Test Loss": cost})
```

Log information to be tracked, passed to web services.

W&B

Do not use parameter sweeps in MC1!



Web GUI with plotting and reporting possibilities. Possibilities for specifying parameter sweeps - hyper parameter optimisation.

Hands-On